

DEV_IMPL_GUIDE - DD: ICN-01 — Internal Communications Service

1. Purpose

This guide explains how to implement DD_ICN-01-Internal_Communications-v1.0.md as executable SOPHIX behavior.

Use it with:

- DD_ICN-01-Internal_Communications-v1.0.md
- DD_SUB-WF-FRAMEWORK-01-Subscription_Workflow_Framework-v1.0.md when this DD participates in subscription workflow operations.
- DD_SUB-WF-FRAMEWORK-01-WORKERS-v1.0.md when this DD references worker topics.

2. Runtime Actors

Runtime component	Responsibility
API pod / module service	Receives requests, validates, loads authoritative state, starts commands.
Owning module database	Stores tables/configuration owned by this DD.
Reference modules	Provide data through approved APIs/client wrappers.
Camunda	Runs BPMN process when this DD is workflow-driven.
Worker apps	Execute logical worker topics.
Drools	Evaluates packages when rules are declared.
Kafka/outbox	Publishes success/failure and integration events.

3. Before Implementation

Confirm these contracts from the DD:

Contract	Value
Endpoint scan reference	DELETE /api/staff-notification-user-channel-identity/{user_id}/{channel}
BPMN/process key	configured BPMN process
Drools package	configured Drools package
Tables	staff_notification, staff_notification_adapter_binding, staff_notification_channel_config, staff_notification_delivery, staff_notification_template, staff_notification_user_channel_identity, staff_notification_user_pref
Events	No explicit events declared

4. Execution Flow

1. Apply migrations and seed configuration/catalog rows.
2. Implement the module service/repository layer around the tables and invariants.
3. Implement public/internal APIs from the DD.
4. Implement rule evaluation if the DD defines Drools packages.
5. Implement event production/consumption through outbox where the DD defines events.
6. Add integration tests for table constraints, API validation, and event payloads.

5. API And Command Handling

Endpoints found in the DD:

- DELETE /api/staff-notification-user-channel-identity/{user_id}/{channel}
- GET /api/groups/kenya-l1-kyc/members?operatorCode=WIK
- GET /api/groups/{group}/members
- GET /api/groups/{group}/members?operatorCode=<>
- GET /api/icn/adapters-registry
- GET /api/staff-notification-adapters-bindings/{operator}/{channel}
- GET /api/staff-notification-adapters-bindings?operatorCode=WIK
- GET /api/staff-notification-channel-config?operatorCode=WIK
- GET /api/staff-notification-templates?operatorCode=WIK
- GET /api/staff-notification-user-channel-identity/{user_id}
- GET /api/staff-notification-user-channel-identity/{user_id}/{channel}
- GET /api/staff-notification-user-pref/{user_id}
- GET /api/staff-notifications/inbox?status=DISPATCHED&limit=20
- GET /api/staff-notifications/stats?operatorCode=WIK&since=2026-05-01
- GET /api/staff-notifications/{id}
- GET /api/staff-notifications/{id}/ack?recipient={userId}
- GET /api/staff-notifications?source_module=BIL-04&since=2026-05-01
- GET /api/users/{userId}
- GET /api/users/{user_id}
- POST /api/orders/{orderId}/user-tasks/{taskId}/complete
- POST /api/staff-notification-adapters-bindings/{operator}/{channel}/test
- POST /api/staff-notification-templates
- POST /api/staff-notifications
- POST /api/staff-notifications/notif_01HZ_KYC_L1_JANE/ack
- POST /api/staff-notifications/{notification_id}/ack
- POST /engine-rest/task/{id}/complete
- PUT /api/staff-notification-adapters-bindings/{operator}/{channel}
- PUT /api/staff-notification-channel-config/{operatorCode}
- PUT /api/staff-notification-templates/{operator}/{code}/{channel}/disable
- PUT /api/staff-notification-user-channel-identity/{user_id}/{channel}
- PUT /api/staff-notification-user-pref/{user_id}

Implementation rules:

- Resolve actor user and role from auth context, not from public request body.
- Check idempotency before inserting a new operation or creating a side effect.
- Return the existing operation/result for the same idempotency key and same request hash.
- Return 409 IDEMPOTENCY_KEY_REUSED for the same key with a different request hash.
- Use transactions or unique constraints for concurrency-sensitive creation.

6. Data Loading

Load data in this order:

1. Authoritative aggregate state from the owning module.

2. Operator/module config from this DD's config tables.
3. Catalog/reference data from approved APIs or client wrappers.
4. Current in-flight operation state from the framework, where relevant.

Do not read current mutable state from cache for state-changing operations.

7. Drools Configuration

Packages found:

- No Drools package explicitly declared.

For each package:

- Define fact classes that mirror the request, current state, config, and reference data.
- Keep rule output as explicit decision fields.
- Unit-test accept and reject cases from the DD.
- BPMN should re-check critical rules after start if state may have changed.

8. BPMN And Worker Topics

Worker topics found:

- adjustment-approval-needed
- auto-resolved-from-auth
- await-l1-kyc-approval
- aws-ses
- bounce-poller
- customer-complaint-routed-for-attention
- directory-sync
- dunning-l2-escalation-internal
- dunning-l3-escalation-internal
- email-adapter
- finance-refunds-approvers
- inapp-adapter
- inapp-firebase-cloud-messaging
- inapp-server-sent-events
- inapp-websocket-fanout
- install-wo-awaiting-dispatcher
- install-wo-awaiting-tech-claim
- install-wo-finalize-confirm-needed
- kenya-dunning-l3-reviewers
- kenya-kyc-leaders
- kenya-l1-kyc
- kyc-final-approval-needed
- kyc-l1-approval-needed
- kyc-rejected-resubmit
- mailgun-api
- microsoft-graph-mail

- msteams-adapter
- msteams-graph-channel
- msteams-incoming-webhook
- network-incident-confirmation-needed
- noc-team
- outage-credit-batch-ready
- pagerduty-events-v2
- postmark-api
- provisioning-failure-noc-investigate
- refund-approval-needed
- sendgrid-api
- shifting-disconnect-confirm
- shifting-reconnect-tech-needed
- slack-adapter
- slack-bot-api
- slack-connect-api
- smtp-classic
- sophix-superadmin
- support-wo-tech-needed-contractor
- support-wo-tech-needed-internal
- twilio-programmable-chat
- vip-customer-escalation
- whatsapp-business-api

Implementation rules:

- Model service tasks with `camunda: type="external"` and the exact topic string.
- Worker handler must validate required variables before calling downstream services.
- Store outputs as process variables with stable names.
- Use message correlation keys exactly as specified by the DD.
- Add compensation paths for each external side effect.

9. Events

Events found:

- No event explicitly declared.

Event rules:

- Emit success only after authoritative commit.
- Emit rejected/failure event only after compensation/final failure decision.
- Use the outbox table/dispatcher; do not publish directly inside the transaction boundary unless the platform standard allows it.

10. Smoke Tests

Add tests for:

- Happy path.

- Invalid role or trigger.
- Invalid state.
- Missing config/catalog row.
- Dependency failure.
- Retry/idempotency.
- Event payload correctness.
- Worker topic failure and compensation.

11. Common Mistakes

- Treating worker-topic names as shell commands.
- Querying another module's database directly.
- Reading current state from cache in a state-changing flow.
- Emitting success before final commit.
- Hiding manual recovery paths for partial external side effects.
- Creating one deployment per topic when a consolidated worker app would be easier to operate.